

Mini-projet 12 — Compréhensions avancées

Pourquoi ce mini-projet ?

Tu maîtrises déjà les compréhensions simples (MP01). Là on monte d'un cran : filtrer **et** transformer, combiner deux listes (toutes les paires **ip x port** à scanner), aplatisr des résultats imbriqués. Ce sont les gestes qui rendent ton code court et lisible — **quand ils restent lisibles** (on verra la limite).

1. Rappel : le filtre if (à la fin)

```
pairs = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8] - le if à la FIN garde ou jette un élément
```

2. if/else ternaire (au début) — transformer, pas filtrer à ne pas confondre

Si tu veux **transformer** chaque élément selon une condition (et non en jeter), le **if/else** va **au début**, dans l'expression :

```
etats = ["bas" if x < 1024 else "haut" for x in [22, 8080, 443]]
# ['bas', 'haut', 'bas'] - chaque élément est REMPLACÉ, aucun n'est jeté
```

La distinction qui piège tout le monde :

Écriture	Position	Effet
[x for x in L if cond]	if à la fin	filtre (garde ou jette)
[a if cond else b for x in L]	if/else au début	transforme (remplace chaque élément)

Repère : un **if sans else** = filtre (fin). Un **if...else** = ternaire de transformation (début).

3. Compréhension imbriquée : combiner deux listes notion clé

Deux **for** l'un après l'autre produisent **toutes les combinaisons** (produit cartésien). Parfait pour « chaque IP x chaque port » :

```
ips = ["10.0.0.5", "10.0.0.6"]
ports = [22, 80]

paires = [(ip, port) for ip in ips for port in ports]
# [('10.0.0.5', 22), ('10.0.0.5', 80), ('10.0.0.6', 22), ('10.0.0.6', 80)]

[ expression for ip in ips for port in ports ]
```

L'ordre de lecture = l'ordre des boucles imbriquées. Le 1er **for** est la boucle **externe**, le 2e la boucle **interne** (celle qui tourne le plus vite). C'est exactement :

```
# équivalent, en boucles classiques :
paires = []
for ip in ips:           # externe
    for port in ports:    # interne
        paires.append((ip, port))
```

4. Aplatisr une liste de listes

Même mécanisme, pour transformer **[[1, 2], [3, 4]]** en **[1, 2, 3, 4]** :

```
listes = [[1, 2], [3, 4], [5]]
plat = [x for sous_liste in listes for x in sous_liste]
# [1, 2, 3, 4, 5]
```

Astuce mémo : lis de gauche à droite comme les boucles empilées : « pour chaque sous_liste dans listes, pour chaque x dans sous_liste, prends x ».

5. Expression génératrice : la version économe (parenthèses)

Rappel du MP11 : remplace les `[ ]` par des `( )` et tu obtiens un **générateur** — utile quand tu enchaînes direct dans un `sum`, `max`, `any` ... sans construire de liste intermédiaire.

```
ports = [22, 80, 443, 8080]
# combien de ports "bien connus" ? — pas besoin de liste, un générateur suffit
nb = sum(1 for p in ports if p < 1024) # 3

# y a-t-il au moins un port ouvert > 1024 ?
any(p > 1024 for p in ports) # True
```

Tu l'as déjà utilisé sans le savoir : le `sum(1 for r in ... if ...)` du MP08, c'est une expression génératrice passée directement à `sum`.

6. ⚠ Le piège : trop imbriqué = illisible

Une compréhension à 3 `for` + 2 `if` tient sur une ligne... mais devient **incompréhensible**. La règle pro : si tu ne peux pas la lire à voix haute d'un trait, **reviens à une boucle classique**. Le but d'une compréhension est la **clarté**, pas la performance de compression. Un code lisible > un code « malin ».

🎯 Ta mission

Objectif : trois fonctions, une par technique.

- `toutes_cibles(ips, ports)` → renvoie la liste de **tous les couples** `(ip, port)` (compréhension imbriquée, double `for`).
- `classer_ports(ports)` → renvoie un **dict** `{port: "bien connu" si < 1024 sinon "haut"}` (compréhension de dict + `if/else` ternaire).
- `aplatir(listes)` → transforme une liste de listes en une seule liste plate.

Tests à faire passer :

```
print(toutes_cibles(["10.0.0.5", "10.0.0.6"], [22, 80]))
# -> [('10.0.0.5', 22), ('10.0.0.5', 80), ('10.0.0.6', 22), ('10.0.0.6', 80)]

print(classer_ports([22, 8080, 443]))
# -> {22: 'bien connu', 8080: 'haut', 443: 'bien connu'}

print(aplatir([[1, 2], [3, 4], [5]]))
# -> [1, 2, 3, 4, 5]
```

Bonus : avec une **expression génératrice** (parenthèses), compte en une ligne combien de couples de `toutes_cibles(...)` ont un port < 1024, sans construire de liste intermédiaire.

✓ Critères de réussite

- `toutes_cibles` produit bien le produit cartésien complet (4 couples pour 2×2).
- `classer_ports` utilise un `if/else` ternaire (au début), pas un filtre.
- `aplatir` renvoie une liste à plat, dans l'ordre.

Si tu bloques — indices progressifs

Étape	Indice
toutes_cibles	<code>[(ip, port) for ip in ips for port in ports]</code> — deux <code>for</code> , l'externe d'abord.
classer_ports	<code>{p: ("bien connu" if p < 1024 else "haut") for p in ports}</code> — accolades + <code>clé: valeur</code> , le ternaire dans la valeur.
aplatir	<code>[x for sous in listes for x in sous]</code> — même schéma que toutes_cibles.

Où ranger : `mini-projets/12-comprehensions-avancees/comprehensions.py` .

Quand c'est prêt : `git add .` → `commit` → `push` , puis « c'est poussé ».